山东大学 计算机科学与技术 学院

云计算技术 课程实验报告

学号：202200130077	姓名：薛润泽	班级：智能 22
实验题目：维吉尼亚密码实现和破解		
实验学时：2	实验日期： 2025.4.16	

实验目的：熟悉维吉尼亚密码算法、了解其安全性。
具体包括：1）实现维吉尼亚密码算法，可以利用任意长度密钥对任意长度句子进行加解密，并在 Docker 中部署；2）通过 Kasiski 和 Friedman 测试方法实现密钥破解。

硬件环境：
计算机一台

软件环境：
Ubuntu

实验步骤与内容：
1、维吉尼亚密码是什么？
维吉尼亚密码是一种多表替换加密算法。维吉尼亚密码的核心原理是通过使用一串密钥，对原文进行不同偏移量的加密。
2、实现维吉尼亚密码算法，设置一串任意长度密钥，设置任意长度的原始句子，通过如下公式对句子进行加密：

$$C_i \equiv P_i + K_i \pmod{26}.$$

其中 Ci 代表加密后文字，Pi 代表加密前文字，Ki 代表当前密钥文字
3、已知维吉尼亚密码的秘钥，对加密后的句子进行解密，通过如下公式对句子进行解密：

$$P_i \equiv C_i - K_i \pmod{26}.$$

4、测试 Kasiski 方法实现密钥长度推测。
（1）Kasiski 用以推测密钥长度

Kasiski 基本原理：多表替换密码的密钥会周期性重复。当明文中相同的字母序列被相同的密钥片段加密时，密文中会出现重复的序列。这些重复序列之间的间隔通常是密钥长度的整数倍，分析这些间隔的因数可推测密钥长度。

Kasiski 方法使用时，要求密文长度要长一些。从而有更多的信息推断密钥长度。

5、测试 Friedman 方法实现密钥长度推测。

Friedman 方法的核心在于重合指数的计算，通过计算重合指数 IC

$$\mathrm{IC} = \frac{\sum_{i=A}^{Z} n_i(n_i - 1)}{N(N-1)}.$$

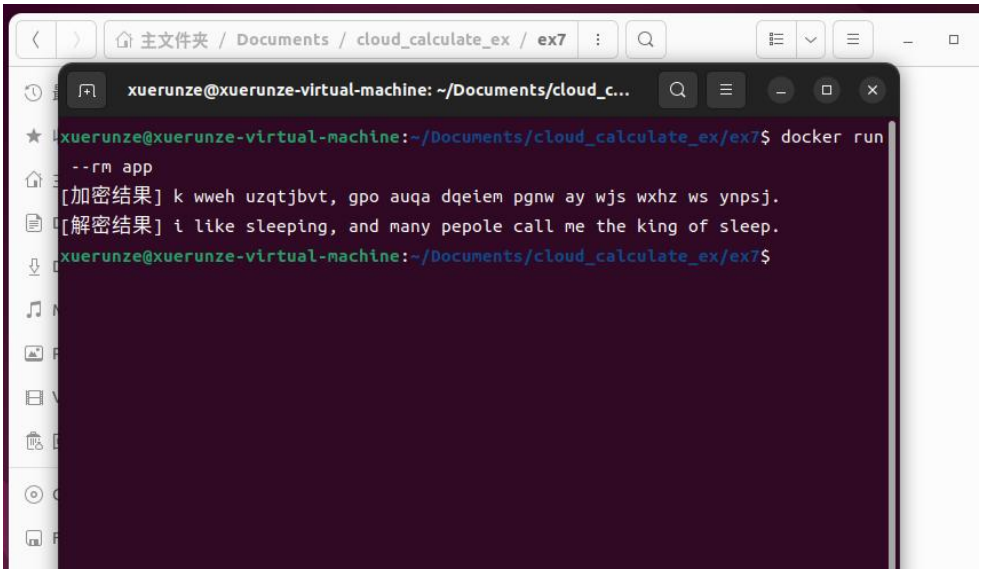
$$n = \frac{m(\mu - \phi)}{\kappa(m-1) + \mu - m\phi},$$

自然语言期望值 IC = 0.065，均匀随机文本 IC = 0.0385 带入上述公式可得

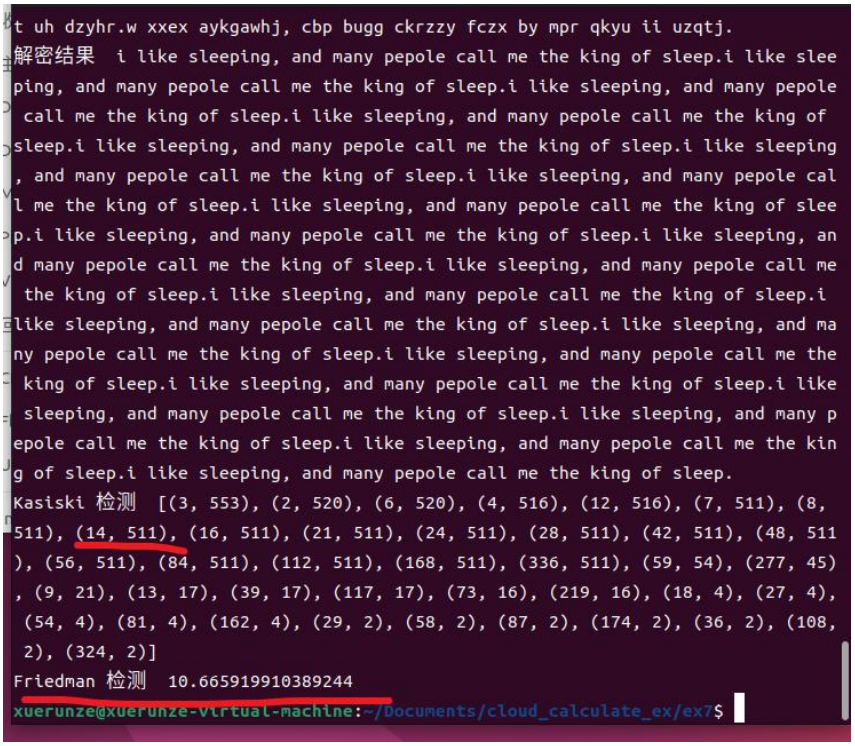
$$k \approx \frac{0.0265\,N}{(0.065 - I) + N(I - 0.0385)}.$$

6、完成实验后进行 docker 镜像部署
构建 docker 镜像： docker build -t app .

结论分析与体会：
原文：content = "i like sleeping, and many pepole call me the king of sleep."
密钥：key = "cloudcomputing"
1、使用 Docker 对维吉尼亚算法进行部署后，加密、解密的结果



2、Kasiski 方法、Friedman 方法实现密钥长度推测：
因为密钥长度推测需要较长文本，我们将上述文本重复 20 次。并进行密钥长度推测：



因为实际密钥是“cloudcomputing”，长度为 14，可以看到 kasiski 检测结果中密钥长度 14 被排在第 8 位，Friedman 检测中密钥长度为 10.66，与 14 接近。

实验代码附加：

```
from math import gcd
from collections import defaultdict, Counter
import re

def kasiski_examination(ciphertext: str, min_sub_len=3) -> list:
    ciphertext = ciphertext.upper().replace(" ", "")
    seq_positions = defaultdict(list)
    for i in range(len(ciphertext) - min_sub_len + 1):
        seq = ciphertext[i:i+min_sub_len]
        seq_positions[seq].append(i)

    repeated_seq = {seq: pos for seq, pos in seq_positions.items() if len(pos) > 1}
    distances = []
    for seq, positions in repeated_seq.items():
        for i in range(len(positions)-1):
            distances.append(positions[i+1] - positions[i])

    possible_key_lengths = Counter()
    for d in distances:
        for factor in range(2, d+1):
```

```
            if d % factor == 0:
                possible_key_lengths[factor] += 1

    return possible_key_lengths.most_common()


def friedman_test(ciphertext: str) -> float:
    ciphertext = re.sub("[^A-Z]", "", ciphertext.upper())
    N = len(ciphertext)
    if N <= 1:
        return 0
    frequency = Counter(ciphertext)
    ic = sum(v * (v-1) for v in frequency.values()) / (N*(N-1))
    estimated_key_length = (0.027 * N) / ((N - 1) * ic - 0.038 * N + 0.065) if ic !=
0 else 0
    return estimated_key_length



content1 = "i like sleeping, and many pepole call me the king of sleep."
content = content1 * 20
key = "cloudcomputing"


# 加密过程
encryption = ""
index = 0
for e in content:
    if ord(e) - ord('a') < 0 or ord(e) - ord('a') > 25:
        encryption += e
        continue
    encryption += chr((ord(e) - ord('a') + ord(key[index]) - ord('a')) % 26 + ord('a'))
    index = (index + 1) % len(key)


# 解密过程
decrypt = ""
index = 0
for e in encryption:
    if ord(e) - ord('a') < 0 or ord(e) - ord('a') > 25:
        decrypt += e
        continue
    decrypt += chr((26 + ord(e) - ord(key[index])) % 26 + ord('a'))
    index = (index + 1) % len(key)
```

```
# 输出结果
print("原始文本 ", content )
print("密钥 ", key)
print("加密结果 ", encryption)
print("解密结果 ", decrypt)
print("Kasiski 检测 ", kasiski_examination(encryption))
print("Friedman 检测 ", friedman_test(encryption))
```